UNIVERSITÄT BERN

3D Geometry Processing
Exercise 03: Geometry Representations

Submission deadline: 17.03.2026, 13:00

Late submissions are not accepted

Submission rules

You should submit a single `.zip` compressed file named `Ex`n`-GroupMemberNames.zip` where n is the number of the current exercise sheet. *Please order the names alphabetically by last name to make life easier for us :)* The file should contain:

- In programming exercises: all code files you changed, and **only** the files you changed (headers and source).
- A readme file (`.txt`, `.pdf` or `.md`) file with the following contents:
  - Names of all team members,
  - Any difficulties encountered and how you resolved them,
  - Optionally any comments about the exercise.
  - (Only in case you used any AI assistants:) List for which purposes/problems you used them, and your judgement of where this was helpful, useless, or misleading.
- (Only in case you used any AI assistants:) Include all your chat transcripts as separate files in an archive `AI-transcripts.zip` inside the main `.zip`.

Theory Exercise (2 pts)

1.1 Derive a signed distance function for the following metric subspace of $\mathbb{R}^2$:

$$S := \{(x, y) \in \mathbb{R}^2 \mid y > x\}$$

1.2 For a curve $f : [a, b] \to \mathbb{R}^n$, we consider

$$T_f(t) = \frac{\frac{\mathrm{d}f}{\mathrm{d}t}}{\|\frac{\mathrm{d}f}{\mathrm{d}t}\|} \tag{1}$$

its *tangent direction* at parameter t.

Give a parametric representation of a planar curve $f : [a, b] \to \mathbb{R}^2$ as a pair of *polynomial* functions (one per coordinate) which has a sharp corner at parameter value $s \in [a, b]$. A sharp corner in this context means a non-differentiable point, i.e., a point at which no tangent direction exists. Show that T_f is not defined at s for your curve and cannot be continuously extended to the whole domain.

1.3 Consider recording a GPS track when going hiking. From experience you may know that increasing recording (position sampling) frequency typically results in seemingly longer recorded distance, as your track takes fewer extreme shortcuts when you walk through hairpin turns. Let's investigate this phenomenon mathematically:

Let the sequence $p_k^N = (x(\frac{k}{N}),\ y(\frac{k}{N})), k = 0, \ldots, N$ denote a uniform sampling of a 2D curve $(x(t), y(t)), t \in [0, 1]$. As the number of sampling N increases, does total chord length always increase monotonically (i.e., is non-decreasing)? If yes, give a proof; if no, give a counterexample.

1.4 Construct a sequence of 2D curves $\mathbf{C}_i(t), t \in [0, 1]$ which satisfy:

1. $\lim_{i \to \infty} \mathbf{C}_i(t) = \mathbf{L}(t), \forall t \in [0, 1]$ where $\mathbf{L}(t)$ is a straight line segment in $[0, 1]$
2. The length of $\mathbf{C}_i(t)$ does not converge to the length of $\mathbf{L}(t)$ in $[0, 1]$.

Submit your solutions to the theoretical exercises as a single `.pdf` file.

Coding Exercise: Marching Triangles / Iso-Contouring (2 pts)

In the lecture you have learned about the *Marching Cubes* algorithm to extract iso-surfaces and the *Marching Squares* algorithm to extract iso-contours. In this exercise, you will implement a variant of the latter that is sometimes called *Marching Triangles*.

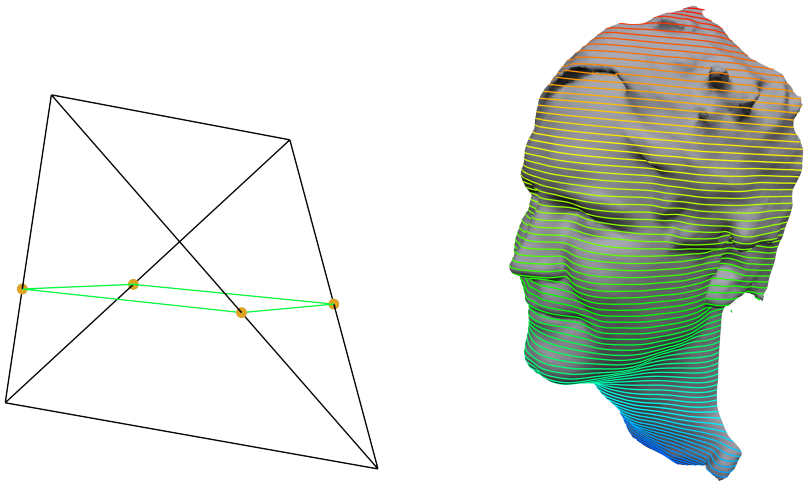

Figure 1: A single Iso-contour for the z coordinate function on a simple tetrahedron generated using the OpenFlipper Primitive generator (left) and a bunch of them on a 3D-scanned face (right).

Goal and algorithm

We start from a triangle mesh $\mathcal{M} = (V, E, F)$ consisting of vertices $V \subset \mathbb{R}^3$, edges $E \subset V \times V$, and faces $F \subset V \times V \times V$. We also have an associated scalar function $f_0 : V \to \mathbb{R}$ defined per-vertex, which we extend to a function $f : M \to \mathbb{R}$ defined on the entire mesh using linear interpolation over edges and faces. **Your task:** Extract the iso-contours where $f = c$ for a given constant $c \in \mathbb{R}$.

We will make the simplifying assumption that $f_0(v) \neq c$ for every vertex v, which means that the iso-contours never pass exactly through a vertex. Note that we can always slightly randomly perturb the function to ensure that this property holds, while staying arbitrarily close to the input[1].

In this case, the set of iso-contours is the union of 1-dimensional line segments (a set of polylines). To compute these, we divide the task into two steps.

Find edge intersections. We test each edge $e = (v, w)$ (canonically parameterized as $e(t) = (1-t) \cdot v + t \cdot w$ for a parameter $t \in [0, 1]$) for an intersection with the iso-contour. In other words, we want to see if there is a parameter t such that $f(e(t)) = c$, i.e., $(1-t) \cdot f_0(v) + t \cdot f_0(w) = c$. If this is the case, we have found a point that must be in the iso-contour, and we can compute its position using t.

Connect the points. Now it only remains to connect these points by adding line segments between pairs of points. To figure out how to do that, consider a single triangle and enumerate all possibilities and which line segments you need to generate in each case.

In our code template, f will simply be the z coordinate of the vertex positions[2]. This simple function makes debugging relatively easy, and it already has useful applications: Given a height map, we can create contour lines of equal elevation for creating a hiking map (cf. Figure 2), or use these curves to fabricate a real-life 3D map with a laser cutter (cf. Figure 3). In 3D printing, a *slicer* is a piece of software that generates tool paths for the extruder nozzle in pretty much the same manner[3].

Implementation

Download `Plugin-Ex03-IsoContouring.zip` from ILIAS and extract it into your `Package-GP` folder. For this exercise you will need to fill in the missing code in the file `IsoContouringPlugin.cc`.

Read the entire supplied code carefully and fill in the missing parts marked as TODOs.

Note that our input mesh is of type OpenMesh::TriMesh, but for the output set of polylines we use an OpenVolumeMesh::PolyhedralMesh. The naming of the latter here may seem super weird, as we are not really working with polyhedral volume meshes, but this is simply a convenient data structure that we just use to output vertices + edges. We will do more work with OpenMesh in the coming weeks and will properly introduce you to its concepts, for now we *hope* that from the given code and comments it will be clear enough how to use it for the required functionality. If not, please don't hesitate to ask, we don't want you to get stuck with such technical issues.

For testing, you can generate simple shapes from OpenFlipper's "Primitives" menu (careful, not

[1] There are more elegant ways where we only symbolically perturb the function, a keyword here would be "simulation of simplicity". This is an advanced topic and not part of course material.

[2] feel free to experiment with different kinds of functions if you like.

[3] Disclaimer: A fully functional slicer may be sliightly more complex, but the basic working principle is the same

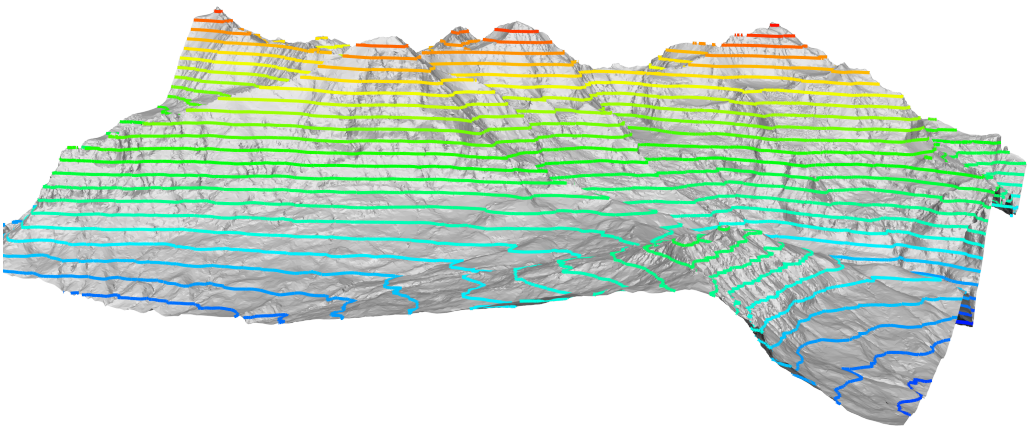

Figure 2: Iso-contours for a height map from the swissALTI3D digital elevation model

every option generates a triangle mesh – other types are ignored by our plugin).

When everything works on the simple shapes, try with your scanned 3D model from the last exercise, and the `eiger-moench-jungfrau.ply` mesh supplied in ILIAS.

We generated the latter from the amazing SwissALTI3D data set (https://www.swisstopo.admin.ch/en/height-model-swissalti3d), but decimated it aggressively to obtain a mesh small enough for fast interactive operations. We'd still recommend only loading these larger meshes into a RelWithDebInfo or Release build of OpenFlipper, in Debug mode you may have to wait a very long time.

Together with your code, submit a few representative screenshots for different types of input.

Figure 3: A DIY model of Eiger + surroundings made by laser-cutting and glueing iso-contours